Lecture 5 revision sheet

Interpolation, regression (curve-fitting) and quadrature

This sheet lists the definitions, formulas and results from Lecture 5, without derivations or discussion. These are in the full notes (`interpolation-regression-quadrature.ipynb` and the homework notebook); the grey pointers give the section. → L*n* marks material that later lectures build on. [*] marks optional material and [**] advanced material in the notes.

Key messages

1. **Interpolation** passes *exactly* through every (exact, distinct-x) data point. **Curve-fitting (regression)** approximates noisy data without passing through every point. **Extrapolation** (evaluating outside the data range) is dangerous: avoid it if possible.
2. The interpolating polynomial of least degree through $N+1$ distinct points is **unique**. Different bases (monomial, Lagrange) give the same polynomial; the basis only changes the cost and robustness of finding it.
3. The monomial basis leads to a **Vandermonde** system, which is ill-conditioned. The **Lagrange** basis needs no matrix solve: the weights are the data values y_i.
4. A single **high-degree** polynomial through **evenly spaced** points can oscillate wildly near the ends (Runge's phenomenon). Use piecewise low-degree interpolants or non-uniform nodes instead.
5. **Least squares** chooses the curve that minimises the sum of squared differences from the data.
6. **Quadrature** approximates a definite integral by summing simple approximations over subintervals. Midpoint and trapezoidal rules are second order; Simpson's rule is fourth order.

1 Interpolation, curve-fitting and extrapolation

Full notes: *Interpolation vs curve-fitting; Extrapolation*

Data	(x_i, y_i), $i = 0, \ldots, N$: that is, $N+1$ points.
Interpolation	Find f with $y_i = f(x_i)$ for all i (the *interpolant*). Assumes exact data at distinct, sorted $x_0 < x_1 < \cdots < x_N$. Used to estimate y for $x \in [\min x_i, \max x_i]$.
Curve-fitting / regression	Find a simpler f that approximates the data (a "cloud of points") but need not pass through any of them. Allows noise and repeated x values; tends to smooth the data. A least-squares straight line is *linear regression*.
Extrapolation	Using f outside the range of the data. Errors can grow rapidly even slightly outside the range; this applies to interpolants and fitted curves alike.
Piecewise	Defined by different formulas on different subintervals; may be continuous with jumps in derivatives (e.g. piecewise linear), or even discontinuous.
Notation	*Degree* of a polynomial = its highest power ($1+x-2x^4$ has degree 4). ":=" means "is defined to be"; "$\equiv$" means "is identically equal to".

Choosing an interpolating/fitting function involves smoothness (how many continuous derivatives), preserving positivity, bounds or periodicity, and cost. Options include polynomials, piecewise polynomials, splines and trigonometric (Fourier-like) series.

2 Polynomial interpolation

Full notes: *Polynomial interpolation; Vandermonde matrix and the monomial basis; Lagrange polynomial*

Uniqueness

- A degree-N polynomial $P_N(x) = a_0 + a_1 x + \cdots + a_N x^N$ has $N+1$ coefficients, matching $N+1$ data points. With distinct x_i there is exactly one solution: **the interpolating polynomial of least degree (at most N) is unique.**
- More coefficients than data (degree $\geq N+1$): *under-determined*, infinitely many solutions. Fewer coefficients than data (degree $\leq N-1$): *over-determined*, generally no exact solution (this is where curve-fitting comes in).
- One point: the constant $P_0 \equiv y_0$. Two points: $a_1 = \dfrac{y_1 - y_0}{x_1 - x_0}$, $a_0 = y_0 - a_1 x_0$.

Monomial basis and the Vandermonde system

Basis $1, x, x^2, \ldots, x^N$ (the *monomials*). Imposing $P_N(x_i) = y_i$ gives

$$V\boldsymbol{a} = \boldsymbol{y}, \qquad V = \begin{pmatrix} 1 & x_0 & \cdots & x_0^N \\ \vdots & \vdots & & \vdots \\ 1 & x_N & \cdots & x_N^N \end{pmatrix} \quad (\texttt{np.vander(xi, increasing=True)}).$$

V is invertible exactly when the x_i are distinct. It is badly **ill-conditioned** (L4): for equally spaced points on $[0, 1]$ its condition number is about 10^8 at $N = 10$ and 10^{17} at $N = 20$, so small data errors give large coefficient errors.

Lagrange form

$$L(x) = \sum_{i=0}^{N} y_i\, \ell_i(x), \qquad \ell_i(x) = \prod_{\substack{m=0 \\ m\neq i}}^{N} \frac{x - x_m}{x_i - x_m}, \qquad \ell_i(x_j) = \begin{cases} 1 & j = i \\ 0 & j \neq i \end{cases}$$

- The weights are the data values y_i, so no linear system has to be solved. By uniqueness, L is the same polynomial as the Vandermonde P_N, just written in a different basis. (For $N = 0$, $\ell_0 := 1$.)
- Linear case ($N = 1$): $L(x) = \dfrac{x - x_1}{x_0 - x_1}\, y_0 + \dfrac{x - x_0}{x_1 - x_0}\, y_1$, the same line as above. Linear interpolation between two values is used again for boundary conditions. → **used in L7**
- Distinct x_i are required: $x_i = x_m$ would divide by zero, and two different y values at the same x cannot both be matched by a function.
- **Python:** in `scipy.interpolate`, `BarycentricInterpolator(xi, yi)` evaluates this polynomial (the older `lagrange` is deprecated and unstable for many points); `np.polyfit(xi, yi, N)` with $N + 1$ points gives the same coefficients, highest power first (reverse of the Vandermonde ordering).

3 Interpolation error

Full notes: *Error in Lagrange interpolation; Aside: The Lagrange Remainder Theorem [**]; A more problematic example [*]*

Lagrange remainder theorem [**]

If f has $N + 1$ continuous derivatives on $[a, b]$ and P_N interpolates f at distinct $x_0, \ldots, x_N \in [a, b]$, then for each $x \in [a, b]$

$$f(x) - P_N(x) = \frac{f^{(N+1)}(c_x)}{(N+1)!}\, \Psi_N(x), \qquad \Psi_N(x) := \prod_{i=0}^{N} (x - x_i),$$

for some c_x between the smallest and largest of $x_0, \ldots, x_N, x$. Hence the **error bound**

$$|f(x) - P_N(x)| \leq \frac{M}{(N+1)!}\, |\Psi_N(x)|, \qquad M := \max_{[a,b]} |f^{(N+1)}|.$$

- The error is zero at the nodes ($\Psi_N(x_i) = 0$), and zero everywhere if f is a polynomial of degree $\leq N$. It is the analogue of the remainder in a truncated Taylor series (L1).
- The result needs f to be smooth. Functions with larger $(N + 1)$-th derivatives tend to have larger errors. For a given f and N, the only thing we control is Ψ_N, i.e. the **choice of nodes**.
- Example $f = x^3$ on $[0.5, 3.5]$: $M = 36.75,\ 21,\ 6$ for $N = 0, 1, 2$. For $N = 2$ the bound equals the actual error, because $f''' \equiv 6$ is constant.
- **Runge's phenomenon** [*]: $f(x) = 1/(1 + 25x^2)$ on $[-1, 1]$ with equally spaced nodes. The fit improves in the middle but oscillates more and more near the ends as N grows (maximum error about 0.4, 1.9 and 60 for $N = 5$, 10, 20). Two causes: $f^{(N+1)}$ grows very rapidly with N, and with equally spaced nodes $|\Psi_N|$ is much larger in the outer subintervals. Non-uniform nodes (e.g. Chebyshev) reduce Ψ_N there.

Take-home message: avoid a single high-degree interpolating polynomial, especially with evenly spaced nodes. Extrapolating such a polynomial is even worse.

4 Piecewise and other interpolants [*]

Full notes: *Piecewise polynomial interpolation [*]; Compare interpolation methods; Other methods [**]*

- **Piecewise linear** on $[x_i, x_{i+1}]$: $P_1^i(x) = \dfrac{y_{i+1}-y_i}{x_{i+1}-x_i}(x-x_i)+y_i$. This is what `plt.plot` draws between data points (`scipy.interpolate.interp1d`).
- **Piecewise quadratic**: a quadratic through each consecutive set of three points (needs an odd number of points). Piecewise interpolants avoid high degree but lose smoothness (corners).

Method	Smoothness	Notes
Lagrange (single polynomial)	C^∞	Can oscillate between and beyond the data.
Piecewise linear	C^0 (corners)	Stays within the data bounds.
Cubic spline [**]	C^2	A cubic on each interval; `scipy.interpolate.CubicSpline`. Can overshoot.
PCHIP [**]	C^1	Piecewise cubic Hermite. Monotone on each interval, so it stays between the two neighbouring y values (no overshoot); second derivative can jump. `scipy.interpolate.PchipInterpolator`.

5 Curve-fitting: least squares regression

Full notes: *Curve-fitting or regression; Least squares error calculation* → **used in L6, L8**

choose P (e.g. a degree-d polynomial) to minimise $\quad E = \sum_{i=0}^{N} \big(P(x_i) - y_i\big)^2.$

- Squaring stops positive and negative errors cancelling. This is minimising the two-norm of the misfit; other norms give different fits and are less or more sensitive to outliers (L4).
- For $d < N$ this is the least-squares solution of the over-determined Vandermonde system $V\boldsymbol{a} \approx \boldsymbol{y}$.
- **Python:** `p = np.poly1d(np.polyfit(xi, yi, d))`; coefficients are returned *highest power first* (for $d = 1$: gradient, then intercept).
- E decreases as d increases. With $d = N$ ($N+1$ points) $E = 0$ to machine precision: the fit *is* the interpolant. Fitting every point (including noise) corresponds to *overfitting* in machine learning.
- **Coefficient of determination** [*]: $R^2 = 1 - \dfrac{SS_{\text{res}}}{SS_{\text{tot}}}$, with $SS_{\text{res}} = \sum(P(x_i)-y_i)^2$ and $SS_{\text{tot}} = \sum(y_i - \bar{y})^2$. For a straight-line fit this equals the square of the r-value from `scipy.stats.linregress`.
- **Power laws** [*]: fit a straight line to $(\log x_i, \log y_i)$; the slope is the exponent. The same idea gives convergence orders from log–log error plots.
- A least-squares fit by a *constant* gives the *mean* of the data. → **used in L8**

6 Quadrature

Full notes: *Quadrature (very brief introduction); The Midpoint rule; Trapezoidal rule; Simpson's rule* → **used in L6–L8**

Set-up

- **Quadrature** = numerical evaluation of a *definite* integral $I := \int_a^b f(x)\,dx$ (the area under the curve).
- Fundamental theorem of calculus: $I = F(b) - F(a)$ for any anti-derivative F ($F' = f$); the constant of integration cancels. Additivity: $\int_a^b = \int_a^c + \int_c^b$, so $I = \sum_{i=0}^{n-1} \int_{x_i}^{x_{i+1}} f\,dx$ with $a = x_0 < x_1 < \cdots < x_n = b$.
- **Notation:** n = number of subintervals (bins); N = polynomial degree used inside each subinterval. For uniform spacing, $\Delta x = (b-a)/n$.
- Two use cases: integrating given *data* (fixed, possibly uneven points), or a *function* we can evaluate anywhere but which may be expensive, so the number of evaluations matters.

Basic rules

Rule	On one interval $[a,b]$, midpoint c	Degree of precision	Order	Interpolant used
Midpoint	$(b-a)\,f(c)$	1	$\mathcal{O}(\Delta x^2)$	constant $f(c)$
Trapezoidal	$(b-a)\,\dfrac{f(a)+f(b)}{2}$	1	$\mathcal{O}(\Delta x^2)$	linear through a, b
Simpson	$\dfrac{b-a}{6}\big(f(a)+4f(c)+f(b)\big)$	3	$\mathcal{O}(\Delta x^4)$	quadratic through a, c, b

- **Degree of precision** M: the rule is exact for all polynomials of degree $\le M$ but not for some polynomial of degree $M+1$. Simpson is exact for quadratics by construction, and for cubics too because the midpoint is placed

symmetrically. **Order** is the convergence rate of the composite rule as $\Delta x \to 0$: halving Δx divides the error by 4 (midpoint, trapezoidal) or 16 (Simpson).
- The composite rules sum the basic rule over all n subintervals, e.g. $I_M = \sum_{i=0}^{n-1} (x_{i+1} - x_i)\, f\big(\frac{x_i + x_{i+1}}{2}\big)$.

Composite trapezoidal rule and function evaluations

$$I_T = \frac{\Delta x}{2}\Big[f(x_0) + 2f(x_1) + 2f(x_2) + \cdots + 2f(x_{n-1}) + f(x_n)\Big] \qquad \text{(uniform } \Delta x\text{)}.$$

Re-using the interior values needs $n+1$ evaluations instead of $2n$ for the naive interval-by-interval sum. Similarly, Simpson with midpoints needs $2n+1$ evaluations when shared end values are re-used ($3n$ naively). Terminology: "XXX rule" on a single interval; "*composite* XXX rule" over many subintervals, implemented to minimise evaluations.

Midpoint vs trapezoidal, and Simpson as a combination

→ used in L6

- Example $\int_0^\pi \sin x\, dx = 2$ with $n = 5$: midpoint error ≈ 0.033 (1.7%, overestimate), trapezoidal error ≈ 0.066 (3.3%, underestimate). For this concave function the trapezoids always lie below the curve, while midpoint errors partly cancel.
- With $E := I - I_{\text{rule}}$, for smooth f typically $E_T \approx -2E_M$ (e.g. $\int_0^1 x^2 dx = \frac{1}{3}$: $I_M = \frac{1}{4}$, $I_T = \frac{1}{2}$, errors $\frac{1}{12}$ and $-\frac{1}{6}$). Counter-examples exist.
- Cancelling the leading error terms gives **Simpson's rule**: $I_S := \frac{2}{3}I_M + \frac{1}{3}I_T = \frac{b-a}{6}\big(f(a) + 4f(c) + f(b)\big)$. Equivalently, it integrates the quadratic interpolant through a, c, b. With $n = 5$ on the sin example its error is about 1.1×10^{-4}.

(L6 uses the same idea of combining two schemes to cancel errors when deriving the trapezoidal time-stepping scheme.)

Newton–Cotes rules [**]

Full notes: *Homework: Newton–Cotes quadrature rules*

Integrate the Lagrange interpolant through $N+1$ *equally spaced* points:

$$I \approx \int_a^b P_N(x)\, dx = \sum_{i=0}^{N} A_i\, f(x_i), \qquad A_i := \int_a^b \ell_i(x)\, dx.$$

$N = 1$ gives weights $(b-a)\big(\frac{1}{2}, \frac{1}{2}\big)$ (trapezoidal); $N = 2$ gives $(b-a)\big(\frac{1}{6}, \frac{4}{6}, \frac{1}{6}\big)$ (Simpson, with $A_0 = A_2 = h/3$, $A_1 = 4h/3$, $h = (b-a)/2$). High N inherits the Runge problem, so high-order Newton–Cotes rules should be used with care.

Python

`scipy.integrate.trapezoid(y, x)` and `scipy.integrate.simpson(y, x=x)` take *data*. Simpson needs an odd number of points (an even number of intervals): our Simpson rule with n subintervals and their midpoints matches `simpson` on $2n+1$ equally spaced points. (Older SciPy names `trapz` and `simps` no longer exist in current versions.)

7 Testing quadrature code (code verification)

Full notes: *Errors and convergence; Simpson's rule Implementation* **→ used in L6**

1. Integrate a function with a known integral (e.g. $\int_0^\pi \sin x\, dx = 2$) with increasing numbers of subintervals n.
2. Plot error against n on log–log axes. The slope is -2 for midpoint and trapezoidal and -4 for Simpson (i.e. orders 2 and 4 in Δx). Fit the slope with `np.polyfit` on the logs.
3. Check agreement with a trusted implementation (SciPy) to machine precision, and test on several functions and intervals: one function giving the right answer is not enough.
4. Check exactness on polynomials up to the degree of precision (e.g. x^2 distinguishes Simpson from midpoint/-trapezoidal).